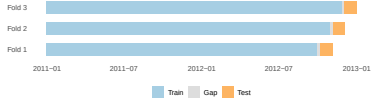
# Applied Machine Learning

## Time Series
## Cross-Validation, Evaluation and Practical Guidance



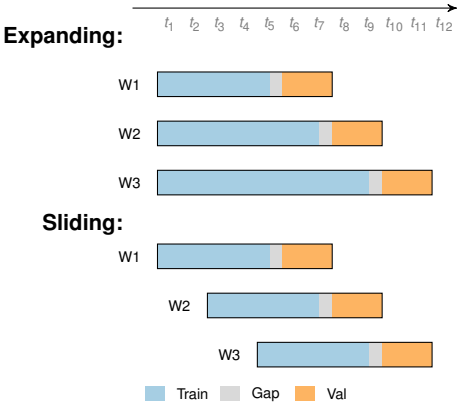

### Learning goals

- Time series cross-validation
- Evaluation metrics and baselines
- Practical guidance and resources



# Time-Series Cross-Validation

# CHRONOLOGY-AWARE SPLITS I




- **Expanding window**: train set grows, validation rolls forward
- **Sliding window**: fixed-width train and validation segments
- Validation window should match the forecasting horizon (e.g., 1 week)
- **Gap**: insert a gap between train and validation to simulate real-world prediction delay

# AVOIDING DATA LEAKAGE I



Time series leakage is subtle and can silently inflate your metrics:

- Compute lagged / rolling features **after** defining the split, or in a way that respects the split boundary
  - Feature computation must be causal and respect the split boundary; non-causal or improperly shifted rolling features can leak future information
- No peeking into future data when standardizing / scaling (fit scaler on train only)
- Never use random shuffled splits for time series $\rightarrow$ use `TimeSeriesSplit` in scikit-learn

# EVALUATION METRICS I



- **MAE** $= \frac{1}{n}\sum_{t=1}^{n}|y_t - \hat{y}_t|$
  - Robust to outliers, easy to interpret (same unit as $y$)
- **RMSE** $= \sqrt{\frac{1}{n}\sum_{t=1}^{n}(y_t - \hat{y}_t)^2}$
  - Penalizes large errors more heavily than MAE
- **SMAPE** $= \frac{100}{n}\sum_{t=1}^{n}\frac{|y_t - \hat{y}_t|}{|y_t| + |\hat{y}_t|}$ (Symmetric Mean Absolute Percentage Error)
  - Scale-independent, useful for comparing across different series
  - Avoids division-by-zero issues of MAPE when $y_t$ is near 0
  - Note: some references use $(|y_t| + |\hat{y}_t|)/2$ in the denominator (ranges 0–200%). The version above ranges 0–100%

# BASELINES I



Always benchmark against naive forecasts. If your model cannot beat these, something is wrong.

- **Naive (persistence) forecast:** $\hat{y}_{t+1} = y_t$ – predict the last observed value
- **Seasonal naive forecast:** $\hat{y}_t = y_{t-s}$ – predict the value from one season ago (e.g., same hour last week: $s = 168$ for hourly data)
- **Moving average:** $\hat{y}_{t+1} = \frac{1}{w} \sum_{i=t-w+1}^{t} y_i$ – average of the last $w$ observations



# Practical Guidance

# SCIKIT-LEARN FOR TIME SERIES I



scikit-learn does not have native time series models (no ARIMA, no built-in lag generation), but provides useful building blocks:

- `TimeSeriesSplit`: chronology-aware CV with configurable `test_size` and `gap` (to simulate real-world prediction lag)
- `FunctionTransformer`: wrap custom feature engineering functions (calendar, cyclic, lags)
- `Pipeline` + `ColumnTransformer`: chain preprocessing and model into a single object
- `HistGradientBoostingRegressor`: fast gradient boosting with native categorical support

# COMMON PITFALLS I



- **Leakage through temporal features**: computing rolling/lag features before the train/test split lets future data leak into training rows
- **Overfitting to rare calendar events**: if your training set has only 2 Christmas periods, the model may memorize rather than generalize holiday effects
- **Ignoring concept drift**: real-world patterns shift over time (e.g., bike usage changed after new stations were added). Periodically retrain or use a sliding window
- **Not inspecting errors over time**: aggregate metrics (MAE, RMSE) can hide that a model is great on weekdays but terrible on weekends. Always plot residuals over time



# Wrap-Up

# KEY TAKEAWAYS I



- Feature engineering often outweighs model complexity in time-series ML
- Calendar, cyclic, lag, and rolling features make temporal patterns accessible to standard ML models
- Use chronology-aware CV (expanding or sliding window) to obtain honest performance estimates
- Always benchmark against a naive or seasonal naive baseline
- Be vigilant about data leakage: compute temporal features *within* each CV fold
- In practice, point forecasts are often not enough – consider quantile regression or conformal prediction for prediction intervals



# Resources

# RESOURCES / FURTHER READING I



- **Hyndman & Athanasopoulos** – “Forecasting: Principles and Practice” (free online textbook)
  `https://otexts.com/fpp3/`
- **scikit-learn** – Time-related feature engineering guide
  `https://scikit-learn.org/stable/auto_examples/applications/plot_cyclical_feature_engineering.html`
- **Kaggle** – Time Series course
  `https://www.kaggle.com/learn/time-series`
- **Cerqueira et al.** (2020) – “Evaluating time series forecasting models: an empirical study on performance estimation methods” (comparison of CV strategies)

# PACKAGES FOR TIME SERIES I



**Python:**

- **statsmodels** – ARIMA, ETS, decomposition
- **StatsForecast** (Nixtla) – fast AutoARIMA, AutoETS
- **Prophet** (Meta) – multiple seasonality, holidays
- **sktime** – sklearn-compatible forecasting framework
- **Darts** (Unit8) – ARIMA to deep learning
- **NeuralForecast** (Nixtla) – N-BEATS, PatchTST
- **tsfresh** – automated feature extraction

**R:**

- **forecast** – ARIMA, ETS, STL decomposition
- **fable** (tidyverts) – tidy forecasting framework
- **feasts** (tidyverts) – feature extraction, decomposition
- **tseries** – stationarity tests, GARCH
- **prophet** – R interface to Meta's Prophet
- **modeltime** – tidymodels forecasting framework
- **tsfeatures** – automated feature extraction

Comprehensive Python list:
`https://github.com/MaxBenChrist/awesome_time_series_in_python`